

Healthcare AI assistant
demos.

This project was a learning
exercise for me to understand
industry tools better.

No frontend ui, the purpose is just learning and gaining
familiarity of the concepts.

What XGBoost is It's a gradient boosting algorithm — it builds trees *sequentially*, where each new tree learns from the mistakes of the previous ones. It's not just a decision tree or a random forest; it corrects itself over iterations. That's why it dominates tabular data problems.

Feature importance XGBoost tracks which features it actually used to make splits across all its trees. This tells you which clinical measurements are doing the real work. In a real healthcare setting this matters beyond just accuracy — you want to know *why* the model is making a decision, and whether those reasons make medical sense.

STEP 1: Load & Explore the Dataset

Dataset: Breast Cancer Wisconsin

Patients : 569

Features : 30

Target : 0 = Malignant, 1 = Benign

Class distribution:

Malignant : 212 (37.3%)

Benign : 357 (62.7%)

Sample features (first 3 rows):

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
0	17.99	10.38	122.8	1001.0	0.11840
1	20.57	17.77	132.9	1326.0	0.08474
2	19.69	21.25	130.0	1203.0	0.10960

STEP 2: Train / Test Split

Training samples : 455

Test samples : 114

STEP 3: Train XGBoost Classifier

Model trained successfully.

Trees built : 100

Max tree depth : 4

Learning rate : 0.1

AUC-ROC (Area Under the Receiver Operating Characteristic Curve): Evaluates how well a model separates classes (e.g., positive vs. negative) across all possible threshold settings.

Accuracy: Evaluates the proportion of total correct predictions (True Positives + True Negatives) divided by the total number of samples.

The "Hiring" Analogy:

- **Accuracy:** Hiring everyone who scores over 50% on a test, without knowing if the test was too easy or hard.
- **AUROC:** Evaluating a recruiter by their ability to consistently rank the best candidates higher than poor candidates, regardless of where the hiring cutoff is set

```
=====
STEP 4: Evaluate on Test Set
=====
```

```
Accuracy : 0.9561 (95.61%)
```

```
AUC-ROC : 0.9944 (1.0 = perfect, 0.5 = random guess)
```

```
Classification Report:
```

	precision	recall	f1-score	support
Malignant	0.97	0.90	0.94	42
Benign	0.95	0.99	0.97	72
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

```
=====
STEP 5: 5-Fold Cross-Validation
=====
```

Now we add RAG

We'll need a LLM Model (using a free local ollama model for this showcase)

Embeddings from ollama (converting text to vectors) - the computer can't read it uses numbers.

RAG (Retrieval-Augmented Generation): The process of retrieving data from an external source (like a database or document) and feeding it to an LLM to improve accuracy.

LangChain's Role: It provides the tools, loaders, and orchestration to make RAG easier, including handling document ingestion, embedding, vector database storage, and retrieval.

```
rag_demo.py > ...
1  # rag_demo.py
2  # Demonstrates Retrieval-Augmented Generation (RAG) for healthcare
3  #
4  # Stack:
5  #   - Documents : Riverside Medical Center clinical guidelines (docs/ folder)
6  #   - Embeddings : Ollama (nomic-embed-text) – converts text to vectors locally
7  #   - Vector DB : ChromaDB – stores and searches those vectors
8  #   - LLM       : Ollama (llama3.2) – generates the final answer
9  #   - Framework : LangChain – ties all of the above together
10 #
11 # Before running:
12 #   ollama pull llama3.2
13 #   ollama pull nomic-embed-text
14
```



```

2 # Demonstrates Retrieval-Augmented Generation (RAG) for healthcare
3 #
4 # Stack:
5 #   - Documents : Riverside Medical Center clinical guidelines (docs/ folder)
6 #   - Embeddings : Ollama (nomic-embed-text) - converts text to vectors locally
7 #   - Vector DB : ChromaDB - stores and searches those vectors
8 #   - LLM : Ollama (llama3.2) - generates the final answer
9 #   - Framework : LangChain - ties all of the above together
10 #
11 # Before running:
12 #   ollama pull llama3.2
13 #   ollama pull nomic-embed-text
14
15 import os
16 from langchain_community.document_loaders import DirectoryLoader, TextLoader
17 from langchain_text_splitters import RecursiveCharacterTextSplitter
18 from langchain_ollama import OllamaEmbeddings, OllamaLLM
19 from langchain_chroma import Chroma
20 from langchain_core.prompts import PromptTemplate
21 from langchain_core.output_parsers import StrOutputParser
22 from langchain_core.runnables import RunnablePassthrough
23
24 DOCS_DIR = "docs"
25 DB_DIR = "chroma_db"
26 EMBED_MODEL = "nomic-embed-text"
27 LLM_MODEL = "llama3.2"

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

• (venv) PS C:\Users\Cyber\Desktop\HealthcareAI> python rag_demo.py
=====
STEP 1: Load Clinical Guideline Documents
=====
Documents loaded: 4
- cardiovascular_guidelines.txt (2879 chars)
- diabetes_guidelines.txt (2111 chars)
- hypertension_protocol.txt (2678 chars)
- medication_interactions.txt (2949 chars)

=====
STEP 2: Split Documents into Chunks
=====
Total chunks created : 31
Avg chunk size       : 341 chars

Example chunk (from cardiovascular_guidelines.txt):
-----
ASPIRIN FOR PRIMARY PREVENTION
- Age < 40: Not recommended routinely
- Age 40-70 with high ASCVD risk: May be considered (discuss bleeding risk vs benefit)
- Age > 70: Do NOT initiate aspirin for primary prevention (bleeding risk outweighs benefit)
- Secondary prevention (existing CVD): Low-dose aspirin 81mg daily - continue indefinitely
-----

```

Rag demo notes.

While these text files are simulated, if you got the real guidelines it would work just as good.

Chromadb

```
HealthcareAI/
├─ chroma_db/      ← created automatically on first run
│  └─ ...          (binary files, don't edit these)
├─ docs/
├─ rag_demo.py
├─ xgboost_demo.py
└─ requirements.txt
```

```
=====
STEP 3: Embed Chunks and Store in ChromaDB
=====
```

```
Embedding model : nomic-embed-text (running via Ollama)
Embedding chunks... (this may take ~20-30 seconds on first run)
Vector store ready. 31 vectors stored in 'chroma_db/'
```

What is the metformin dose for a new diabetes patient?

```
=====
STEP 4: Demonstrate Retrieval (Semantic Search)
=====
```

```
Query: "What is the metformin dose for a new diabetes patient?"
```

```
Top 3 retrieved chunks:
```

```
[1] Source: cardiovascular_guidelines.txt
STATIN THERAPY GUIDELINES
```

```
High-intensity statins (reduce LDL by >= 50%):
```

- Atorvastatin 40-80mg daily
- Rosuvastatin 20-40mg daily

```
Indications: Clinical ASCVD, LDL >= 190 mg/dL, diabetes + high risk, 10-year risk >= 20%
```

```
Moderate-intensity statins (reduce LDL by 30-50%):
```

- Atorvastatin 10-20mg
- Ro...

```
[2] Source: medication_interactions.txt
```

```
=====
STEP 7: Additional RAG Queries
=====
```

```
Q: What are the contraindications for metformin?
```

```
A: The context does not explicitly state a list of contraindications for metformin. However, it mentions that metformin should be held 48 hours before surgery or procedures involving contrast dye, as it may increase the risk of lactic acidosis (rare but serious).
```

```
Q: A patient is on warfarin and needs an antibiotic. What should I watch out for?
```

```
A: For a patient on warfarin, you should be aware of the risk of INR doubling or tripling due to the interaction with Fluoroquinolone antibiotics. This can increase the risk of bleeding. You should use an alternative antibiotic if possible, and if unavoidable, reduce the warfarin dose by 30-50% and check INR in 3-5 days.
```

```
Q: What LDL target should a patient with known heart disease aim for?
```

```
A: According to the guidelines, patients with heart disease should aim for an LDL-C target of <=100 mg/dL.
```

We asked for 3 chunks, but only needed 2

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

k too low — risk missing the right chunk

k too high — you flood the LLM with irrelevant context, which can confuse it and slow things down

With RAG or without Rag.

```
=====
STEP 5: WITHOUT RAG - LLM Answering From Memory Only
=====
```

```
Question: "At Riverside Medical Center, what is the preferred diuretic for hypertension treatment and why?"
```

```
LLM response (no context):
-----
```

```
I can't provide information about specific medications or treatments used at a particular hospital. If you're looking for information on the preferred diuretics for hypertension treatment in general, I'd appy to help with that. Would you like some general information?
```

```
=====
STEP 6: WITH RAG - LLM Answering Using Retrieved Context
=====
```

```
Question: "At Riverside Medical Center, what is the preferred diuretic for hypertension treatment and why?"
```

```
LLM response (with RAG context):
-----
```

```
The preferred diuretic for hypertension treatment at Riverside Medical Center is chlorthalidone, with a daily dose of 12.5-25mg. This is preferred over HCTZ due to its medication profile.
```

RAG with a strict prompt ("*answer ONLY from this context*") is important. You're engineering around the hallucination risk rather than trusting the model to self-police.

Now that we have 2 programs we can deploy an agent to decide which one to use

- Tool 1: `search_guidelines` → the RAG pipeline
- Tool 2: `predict_tumor` → the XGBoost model

Demo 1 - just rag

Demo 2 - just xgboost

Demo 3 - both

Issue I ran into — Demo 1 & 2 garbled final answers: The LLM is printing raw JSON tool-call syntax as its final answer instead of natural language. That's llama3.2 3B getting confused about when it's done calling tools.

A bigger parameter model (8 billion) will give better performance.

```
=====
DEMO 1: Query that needs the RAG tool only
=====
```

```
QUERY: What is the HbA1c target for a diabetic patient over 75 years old at Riverside Medical Center?
```

```
→ Agent calls tool : search_guidelines
```

```
Input      : {"__arg1": "HbA1c target for a diabetic patient over 75 years old at Riverside Medical Center"}
```

```
Tool returned : For a diabetic patient over 75 years old, the HbA1c target is < 8.0%....
```

```
FINAL ANSWER:
```

```
This answer was provided by Medscape's HbA1c Guidelines tool.
```

The LLM hallucinated the source name, but got the right answer

```
=====
DEMO 2: Query that needs the XGBoost tool only
=====
```

```
QUERY: A patient has these tumor measurements: {"radius_mean": 17.99, "texture_mean": 10.38, "perimeter_mean": 122.8, "area_mean": 1001.0, "smoothness_mean": 0.1184, "compactness_mean": 0.2776, "concavity_mean": 0.3001, "concave_points_mean": 0.1471, "symmetry_mean": 0.2419, "fractal_dimension_mean": 0.07871}. Is this tumor likely malignant or benign?
```

```
→ Agent calls tool : predict_tumor
```

```
Input      : {"__arg1": "{\"radius_mean\": 17.99, \"texture_mean\": 10.38, \"perimeter_mean\": 122.8, \"area_mean\": 1001.0, \"smoothness_mean\": 0.1184, \"compactness_mean\": 0.2776, \"concavity_mean\": 0.3001, \"concave_points_mean\": 0.1471, \"symmetry_mean\": 0.2419, \"fractal_dimension_mean\": 0.07871}"}
Tool returned : Prediction: Malignant Confidence: 84.8% P(Malignant): 0.8481 P(Benign): 0.1519...
```

```
FINAL ANSWER:
```

```
Based on the tumor measurements provided, the model predicts that the tumor is likely malignant with a confidence level of 84.8%. The probability of the tumor being malignant is 84.81%, while the probability of it being benign is 15.19%.
```

Perfect

Query that needs both tools (this was the hardest one to get working :(

```
=====
DEMO 3: Query that needs both tools
=====
```

```
QUERY: A patient has a suspicious tumor with measurements: {"radius_mean": 13.54, "texture_mean": 14.36, "perimeter_mean": 87.46, "area_mean": 566.3, "smoothness_mean": 0.09779, "compactness_mean": 0.08129, "concavity_mean": 0.06664, "concave_points_mean": 0.04781, "symmetry_mean": 0.1885, "fractal_dimension_mean": 0.05766}. Predict whether it is malignant or benign, and also tell me what the diabetes HbA1c target is for elderly patients at Riverside Medical Center.
```

```
→ Agent calls tool : predict_tumor
  Input           : {"__arg1": "{\"radius_mean\": 13.54, \"texture_mean\": 14.36, \"perimeter_mean\": 87.46, \"area_mean\": 566.3, \"smoothness_mean\": 0.09779, \"compactness_mean\": 0.08129, \"concavity_mean\": 0.06664, \"concave_points_mean\": 0.04781, \"symmetry_mean\": 0.1885, \"fractal_dimension_mean\": 0.05766}"}
  Tool returned    : Prediction: Malignant Confidence: 84.8% P(Malignant): 0.8481 P(Benign): 0.1519...
  Tool returned    : The diabetes HbA1c target for elderly patients (>75 years) at Riverside Medical Center is < 8.0%....
```

```
FINAL ANSWER:
Based on the tool outputs:
```

For the tumor prediction, the model predicts that the patient has a malignant tumor with a confidence level of 84.8%.

For the diabetes HbA1c target, the guidelines recommend an HbA1c target of less than 8.0% for elderly patients (>75 years) at Riverside Medical Center.

```
=====
```

Improvements:

XGBoost

- Hyperparameter tuning with grid search or Optuna
- Handle imbalanced datasets with `scale_pos_weight`
- Save and reload the trained model with joblib instead of retraining every run
- Early stopping to prevent overfitting automatically

RAG

- Increase `k` to retrieve more chunks and improve answer completeness
- Hybrid search (semantic + keyword combined) for better retrieval accuracy
- Re-ranking retrieved chunks before passing to the LLM
- Reload from existing ChromaDB instead of re-embedding on every run
- Swap in real clinical guidelines (ACC/AHA publish PDFs publicly)

Agent

- Upgrade to `llama3.1:8b` for more reliable tool calling
- Add more tools (e.g. a drug dosage calculator, a risk score tool)
- Add a max iterations limit to prevent infinite retry loops like what

LangChain (concepts not yet demonstrated)

- LCEL chains explicitly as a concept
- Prompt templates with variable injection
- Conversation memory so the agent remembers previous questions in a session
- Structured output parsing to get Python dicts back instead of raw text